

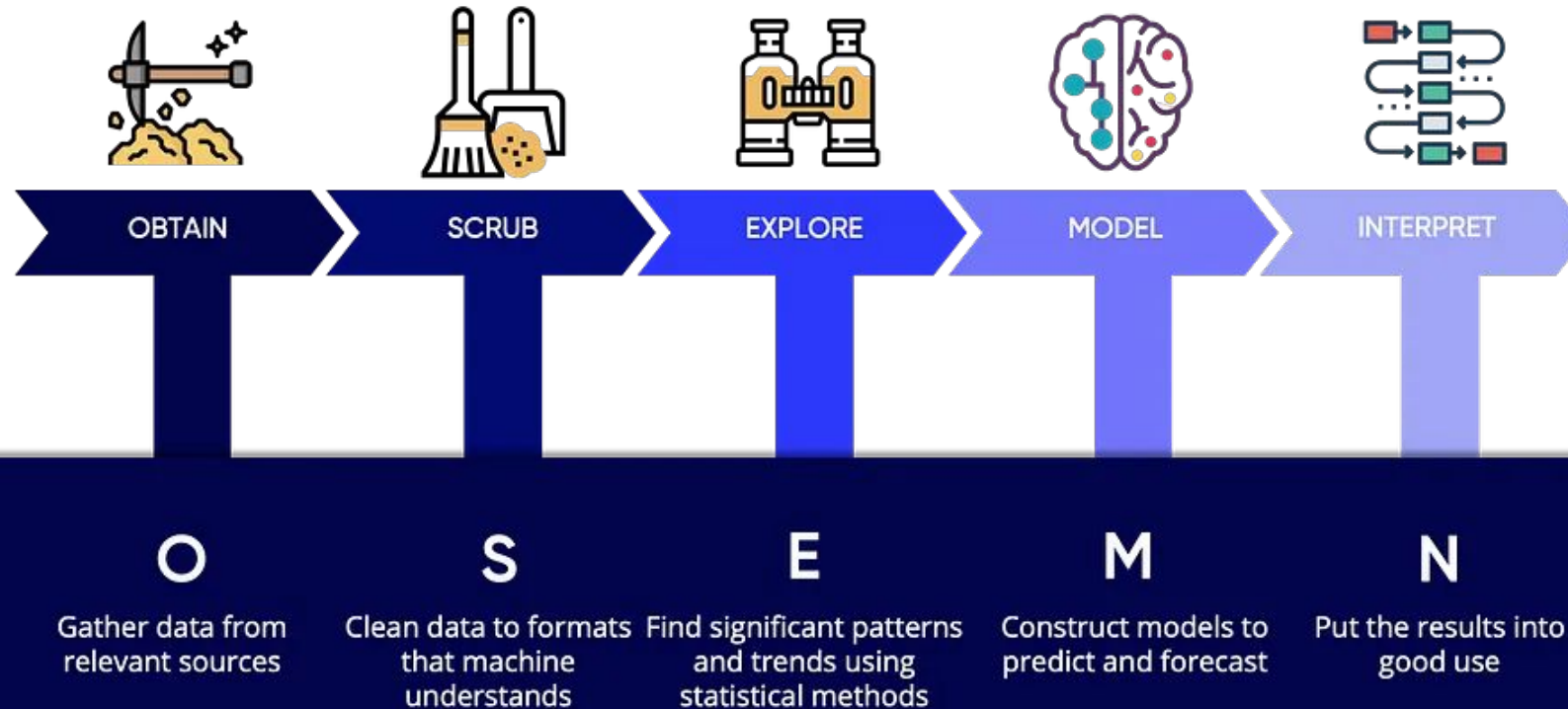
GCI week3

Cleaning Data Using Pandas



Original Lecturer: Ryota Tai

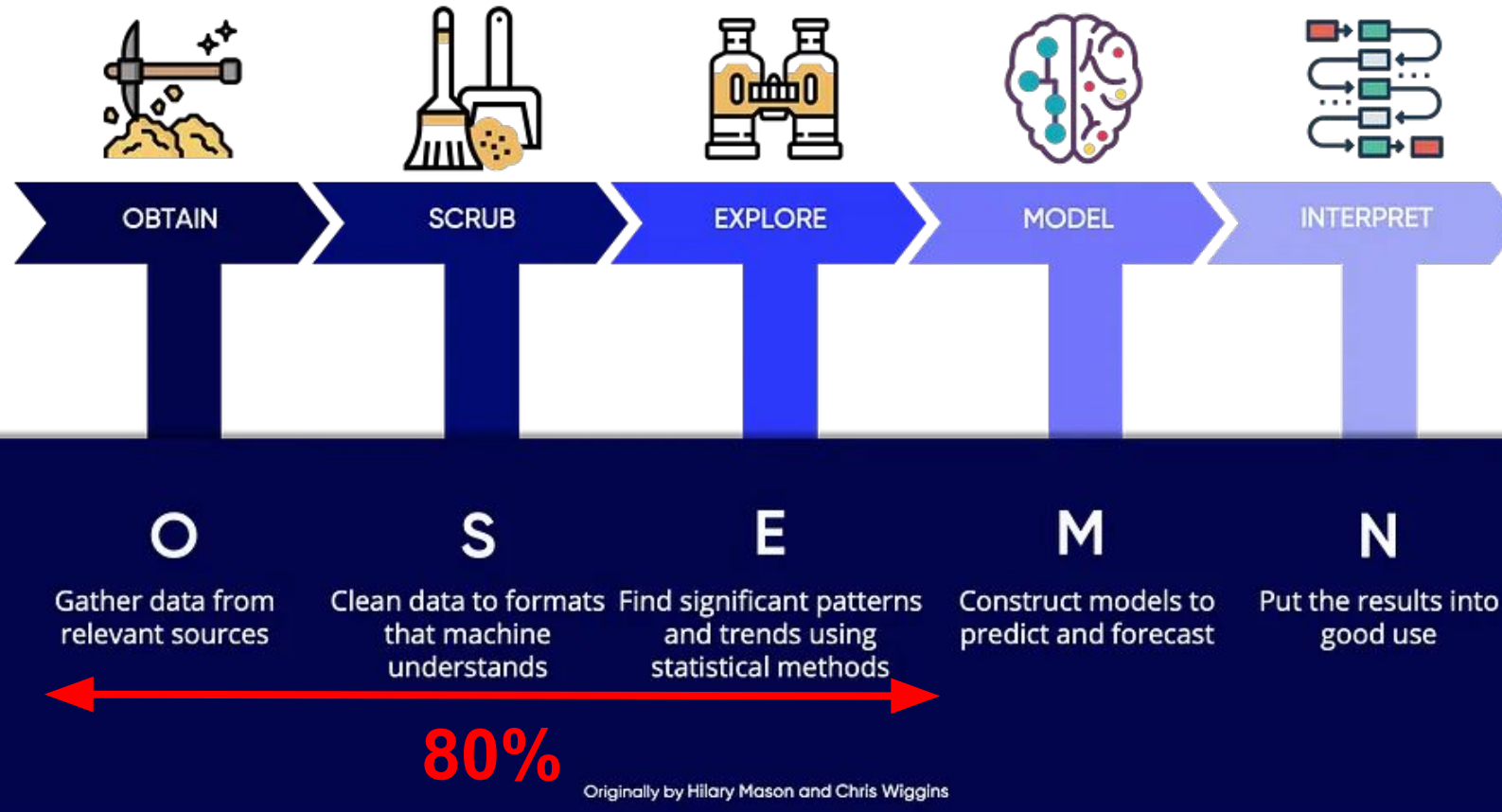
Data Science Process



Originally by Hilary Mason and Chris Wiggins

Reference : <https://towardsdatascience.com/5-steps-of-a-data-science-project-lifecycle-26c50372b492>

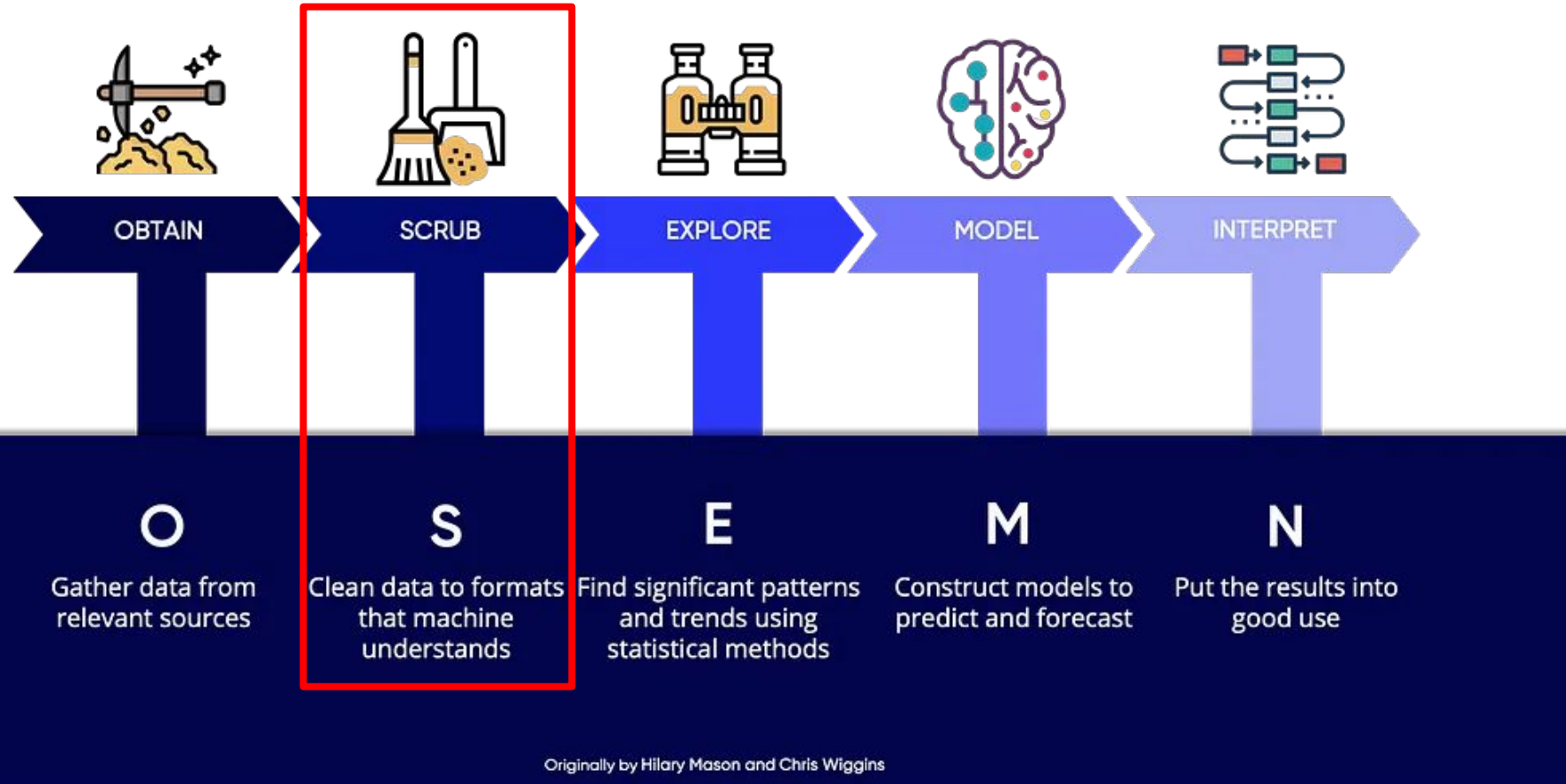
Data Science Process



Reference : <https://towardsdatascience.com/5-steps-of-a-data-science-project-lifecycle-26c50372b492>

**Focus of this
lecture**

Data Science Process



Reference : <https://towardsdatascience.com/5-steps-of-a-data-science-project-lifecycle-26c50372b492>

Structured Data

- Fixed Length file
- Excel/CSV
- RDB Data

Unstructured Data

- Text
- Audio
- Image
- Video
- Sensor Log

Semi-structured Data

- XML
- Json
- html

Tabular



Standardized



Focus of this lecture



- Library that enables data to be handled as a numeric table
- **Unique data structure** enables processing of various types of data
- In short, you can handle data as if you were using Excel

Strengths

- Has methods for processing numerical data, time-series data and character strings
- Strong in statistical processing such as data aggregation
- Reads and writes a variety of file formats, including csv, excel, and json

Weaknesses

- Slow calculation speed
- High memory consumption

Common data structures used in Pandas are Series and DataFrame.

Series (1 dimension)

		ID
Row 1	0	100
Row 2	1	101
Row 3	2	102
Row 4	3	103
Row 5	4	104

DataFrame (2 dimension)

		Column 1	Column 2	Column 3	Column 4
		ID	City	Birth_year	Name
Row 1	0	100	Tokyo	1990	Hiroshi
Row 2	1	101	Osaka	1989	Akiko
Row 3	2	102	Kyoto	1992	Yuki
Row 4	3	103	Hokkaido	1997	Satoru
Row 5	4	104	Tokyo	1982	Steve

Pandas Data Structure



DataFrame is a Numpy 2D array with row and column labels

Row labels are called **index**, Column labels are called **column**.

Series (1 dimension)

		ID
Row 1	0	100
Row 2	1	101
Row 3	2	102
Row 4	3	103
Row 5	4	104
	index	values

DataFrame (2 dimension)

		Column 1	Column 2	Column 3	Column 4	
		ID	City	Birth_year	Name	columns
Row 1	0	100	Tokyo	1990	Hiroshi	
Row 2	1	101	Osaka	1989	Akiko	
Row 3	2	102	Kyoto	1992	Yuki	
Row 4	3	103	Hokkaido	1997	Satoru	
Row 5	4	104	Tokyo	1982	Steve	
	index					values

- Pandas provides a number of methods for working with DataFrame format.

Content and keywords covered in this lecture

Data selection and assignment / Data extraction / Value sorting / Data merging / Data deletion /
Data aggregation and group operations / Hierarchical indexing / Dealing with missing values

- **You do not have to memorize everything**
- Today's goal is to give you a brief idea of what you can do with Pandas.

Let's actually run the code!
Please open the Jupyter Notebook.

Data extraction



Different methods of extracting data are used depending on what is to be **acquired** and **specified**.

Get Series
and
DataFrame

Specifying label					
<u>loc</u> df.loc[1:2, ['ID', 'Birth_year']]					
	ID	City	Birth_year	Name	Score
0	100	Tokyo	1990	Hiroshi	0
1	101	Osaka	1989	Akiko	10
2	102	Kyoto	1992	Yuki	20

Specifying index					
<u>iloc</u> df.iloc[0:2, [0,2]]					
	ID	City	Birth_year	Name	Score
0	100	Tokyo	1990	Hiroshi	0
1	101	Osaka	1989	Akiko	10
2	102	Kyoto	1992	Yuki	20

Get single
element

<u>at</u> df.at[2, 'Birth_year']					
	ID	City	Birth_year	Name	Score
0	100	Tokyo	1990	Hiroshi	0
1	101	Osaka	1989	Akiko	10
2	102	Kyoto	1992	Yuki	20

<u>iat</u> df.iat[2, 2]					
	ID	City	Birth_year	Name	Score
0	100	Tokyo	1990	Hiroshi	0
1	101	Osaka	1989	Akiko	10
2	102	Kyoto	1992	Yuki	20

Data merge

Inner join is performed when a **key exists in both data**.

df1

	id	city	birth_year	name
0	100	Tokyo	1990	Hiroshi
1	101	Osaka	1989	Akiko
2	102	Kyoto	1992	Yuki
3	103	Hokkaido	1997	Satoru
4	104	Tokyo	1982	Steeve

df2

	id	math	english	sex	index_num
0	100	50	90	M	0
1	101	43	30	F	1
2	102	33	20	F	2
3	105	76	50	M	3

【Specify the key】

- can be specified using argument: `on`, `left_on`, `right_on`, `left_index`, `right_index`

- If not specified, the column common to both sets of data is used as the key

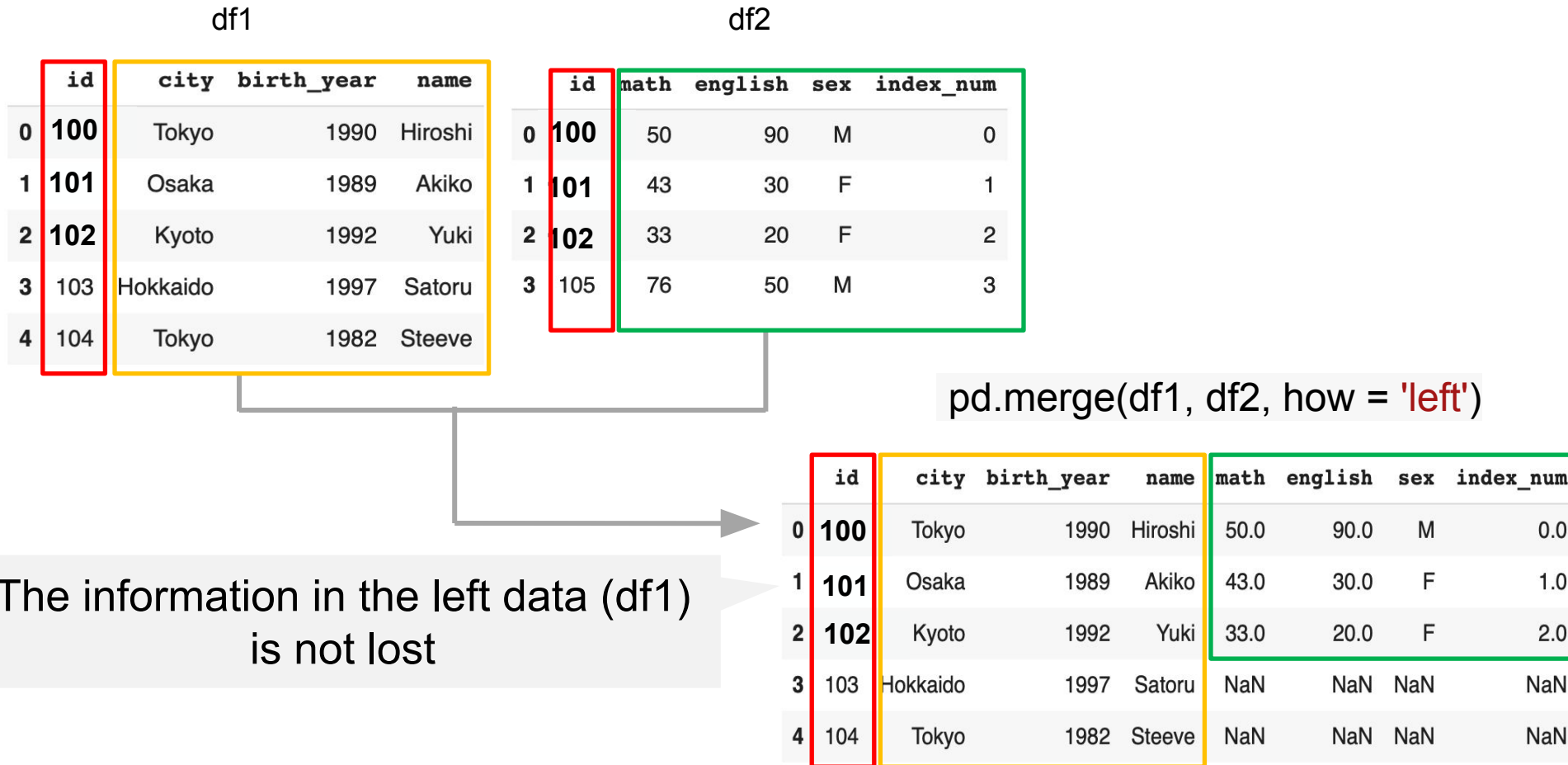
`pd.merge(df1, df2)`

	id	city	birth_year	name	math	english	sex	index_num
0	100	Tokyo	1990	Hiroshi	50	90	M	0
1	101	Osaka	1989	Akiko	43	30	F	1
2	102	kyoto	1992	Yuki	33	20	F	2

Only rows where id (key) exists in both df1 and df2 are merged

Data merge

Left outer join is performed based on **the key of the left data**.



Data merge

Full outer join is operated if either data has a key.

df1

	id	city	birth_year	name
0	100	Tokyo	1990	Hiroshi
1	101	Osaka	1989	Akiko
2	102	Kyoto	1992	Yuki
3	103	Hokkaido	1997	Satoru
4	104	Tokyo	1982	Steeve

df2

	id	math	english	sex	index_num
0	100	50	90	M	0
1	101	43	30	F	1
2	102	33	20	F	2
3	105	76	50	M	3

`pd.merge(df1, df2, how = 'outer')`

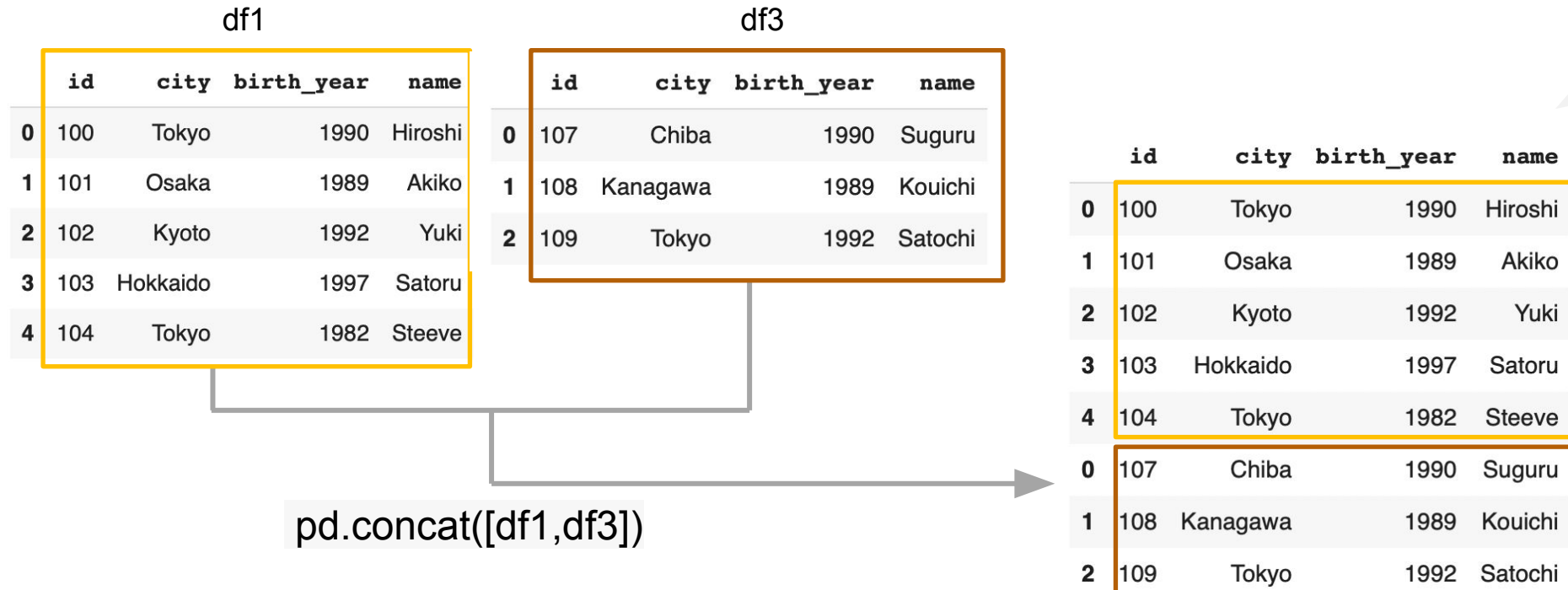
No information is lost in either data

	id	city	birth_year	name	math	english	sex	index_num
0	100	Tokyo	1990.0	Hiroshi	50.0	90.0	M	0.0
1	101	Osaka	1989.0	Akiko	43.0	30.0	F	1.0
2	102	Kyoto	1992.0	Yuki	33.0	20.0	F	2.0
3	103	Hokkaido	1997.0	Satoru	NaN	NaN	NaN	NaN
4	104	Tokyo	1982.0	Steeve	NaN	NaN	NaN	NaN
5	105	NaN	NaN	NaN	76.0	50.0	M	3.0

Data merge(concat)



Vertical join stacks and concatenates the data without specifying a key.



Stacking
using
common
columns

Data merge(concat)

When column names are different in **vertical join**, the areas with missing data are missing values.

df1

	id	city	birth_year	name
0	100	Tokyo	1990	Hiroshi
1	101	Osaka	1989	Akiko
2	102	Kyoto	1992	Yuki
3	103	Hokkaido	1997	Satoru
4	104	Tokyo	1982	Steeve

df2

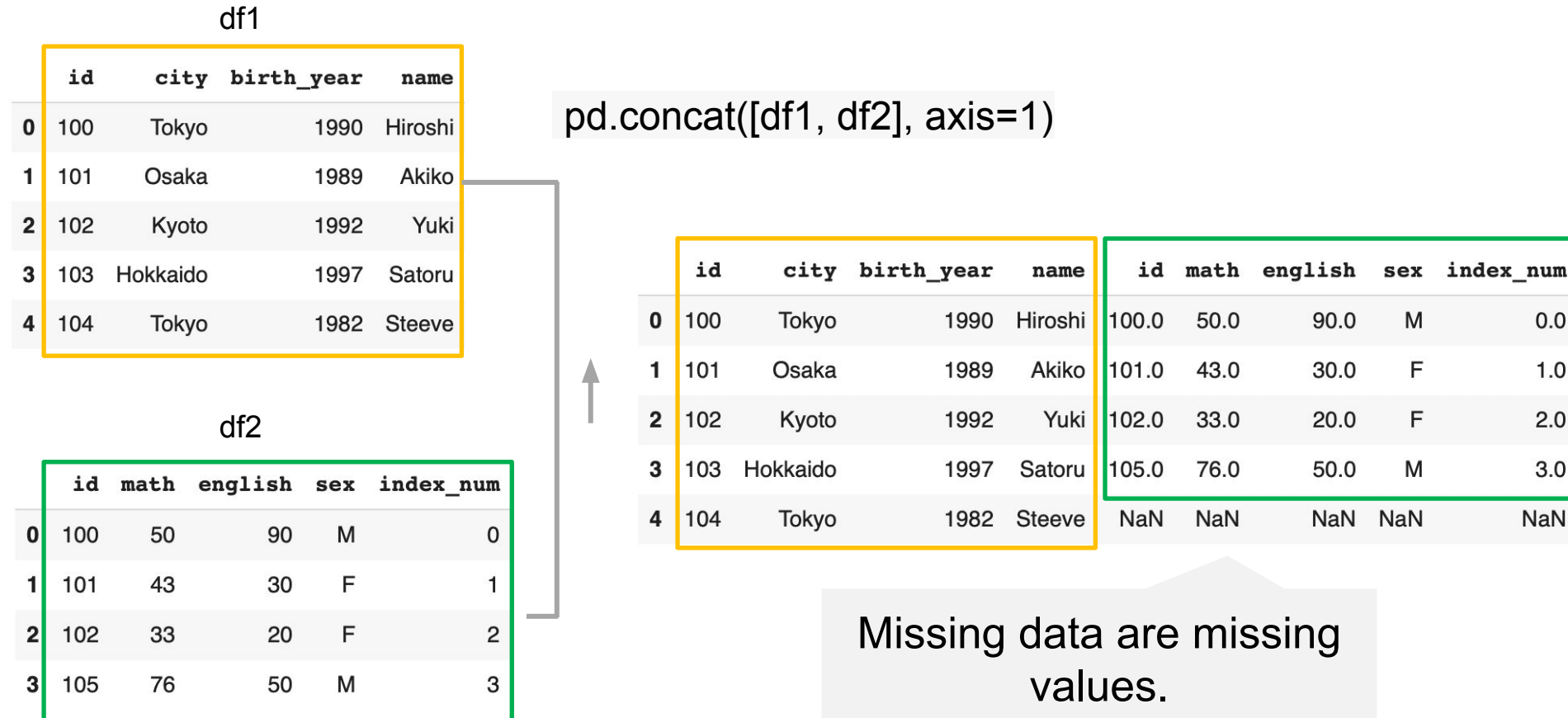
	id	math	english	sex	index_num
0	100	50	90	M	0
1	101	43	30	F	1
2	102	33	20	F	2
3	105	76	50	M	3

`pd.concat([df1, df2], sort=True)`

	birth_year	city	english	id	index_num	math	name	sex
0	1990.0	Tokyo	NaN	100	NaN	NaN	Hiroshi	NaN
1	1989.0	Osaka	NaN	101	NaN	NaN	Akiko	NaN
2	1992.0	Kyoto	NaN	102	NaN	NaN	Yuki	NaN
3	1997.0	Hokkaido	NaN	103	NaN	NaN	Satoru	NaN
4	1982.0	Tokyo	NaN	104	NaN	NaN	Steeve	NaN
0	NaN	NaN	90.0	100	0.0	50.0	NaN	M
1	NaN	NaN	30.0	101	1.0	43.0	NaN	F
2	NaN	NaN	20.0	102	2.0	33.0	NaN	F
3	NaN	NaN	50.0	105	3.0	76.0	NaN	M

Data merge(concat)

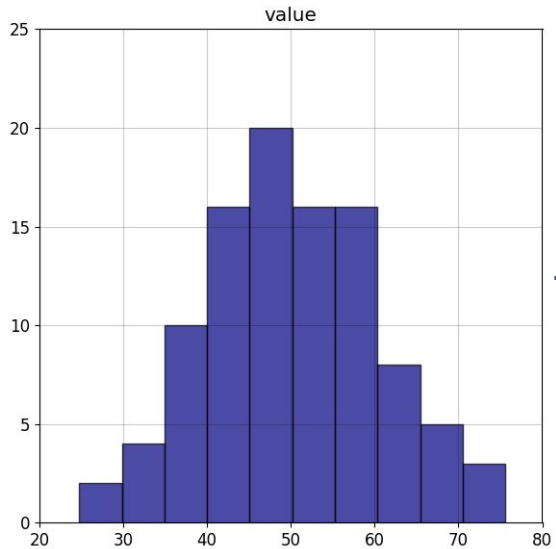
Horizontal join ignores columns and **concatenates data based on index**.



Data binning

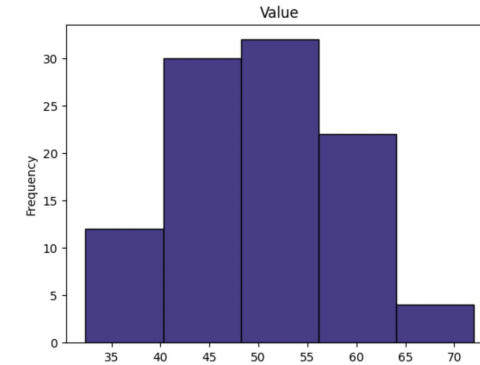
When binning, **divide data equally or in equal numbers.**

```
data = DataFrame({"value":np.random.normal(50,10,100)})  
# random number sampled from a normal distribution
```



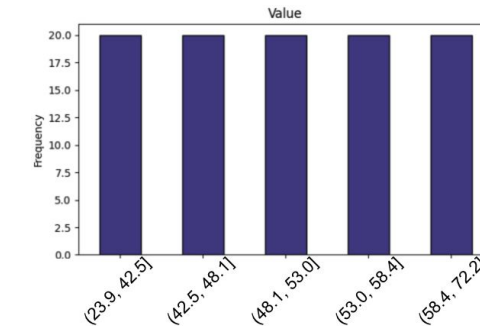
cut(): Split data at **equal interval**

pd.cut(data["value"], bins=5)



qcut(): Split data at **same number of items**

pd.qcut(data["value"], q=5)





Hierarchical Index



Hierarchize indexes and columns to **improve interpretability and make aggregation easy**.

	Store name	Place	Product name	Color	Sales
0	Store A	Osaka	Product 1	Red	200
1	Store A	Osaka	Product 1	Blue	0
2	Store A	Osaka	Product 2	Red	100
3	Store A	Tokyo	Product 1	Red	500
4	Store A	Tokyo	Product 1	Blue	300
5	Store A	Tokyo	Product 2	Red	400
6	Store B	Osaka	Product 1	Red	800

	Product name	Product 1		Product 2	level 0
	Color	Blue	Red	Red	level 1 (level -1)
Store name	Place				
Store A	Osaka	0	200	100	
	Tokyo	300	500	400	
Store B	Osaka		800		
level 0		level 1 (level -1)			

- Improved interpretability without changing the data itself
- Easier filtering and aggregation